AI 답변 검증기

Answer Auditor

AI 답변에서 **다시 확인해 볼 문장**을 찾아
검색된 출처와 함께 표시합니다.

데모 https://ai-verify-deliverability.vercel.app
코드 https://github.com/rhdiddlwpffl/AI_Verify_deliverability



문제

AI 답변은 그럴듯해서, 어디가 위험한지 알기 어렵다

길고 자신 있게 쓰인 답변 속에서 무엇을 먼저 확인해야 할까요?

- 수치, 연도, 인명처럼 **틀리면 곤란한 문장**이 의견과 섞여 있다
- "항상", "반드시" 같은 **단정 표현**은 근거가 약해도 눈에 안 띈다
- 출처 없이 쓰인 인용은 사실처럼 읽힌다
- 전부 다시 검색하기엔 시간이 너무 든다

해결 방법

새 답변을 만들지 않고, 받은 답변을 검사한다

답변을 붙여 넣으면 문장으로 나누고, 확인이 필요한 문장과 근거가 약한 부분을 표시합니다.

다른 AI와의 차이

또 하나의 답변을 생성하지 않습니다. 이미 받은 답변의 **취약한 지점**을 찾아 줍니다.

핵심 원칙

근거가 있다고 참은 아니다. 그래서 "참/거짓"으로 판정하지 않고 "검색된 출처 기준"의 참고 정보로만 보여 줍니다.

> 최종 판단은 사용자가 원문 출처를 직접 확인해서 합니다. 화면 어디에나 이 안내를 둡니다.

사용 흐름

붙여 넣고, 표시를 보고, 근거를 확인한다

로그인 없이 바로 사용합니다. 저장은 사용자가 체크했을 때만 합니다.

① 붙여 넣기

AI 답변 최대 5,000자

② 문장 분리

문장 위치를 정확히 기억해 원문에 하이라이트

③ 확인 대상 표시

사실 확인이 필요한 문장과 취약 신호(수치, 단정 표현 등)

④ 근거 비교

위키백과 문서와 비교해 상태와 인용문 표시

결과 표시

상태 이름에 "참/거짓"이 없다

상태	의미
검색된 출처에서 뒷받침하는 내용 발견	참이라는 뜻이 아님
검색된 출처와 상충하는 내용 발견	거짓이라는 뜻이 아님
출처 간 엇갈림	출처마다 말이 다름
근거 부족	관련 출처를 못 찾았거나 판단 불가
검색 실패	"근거 부족"과 구분해서 표시
검증 대상 아님	의견, 인사, 질문 등

동작 원리

AI는 근거를 가져오고, 최종 상태는 코드가 계산한다

문장 분리
규칙 기반. 위치를 정확히 계산

▶

분류
Claude Haiku가 확인 대상과 검색어 결정, 규칙으로 취약 신호 추가

▶

검색
위키백과 API (키와 카드 불필요)

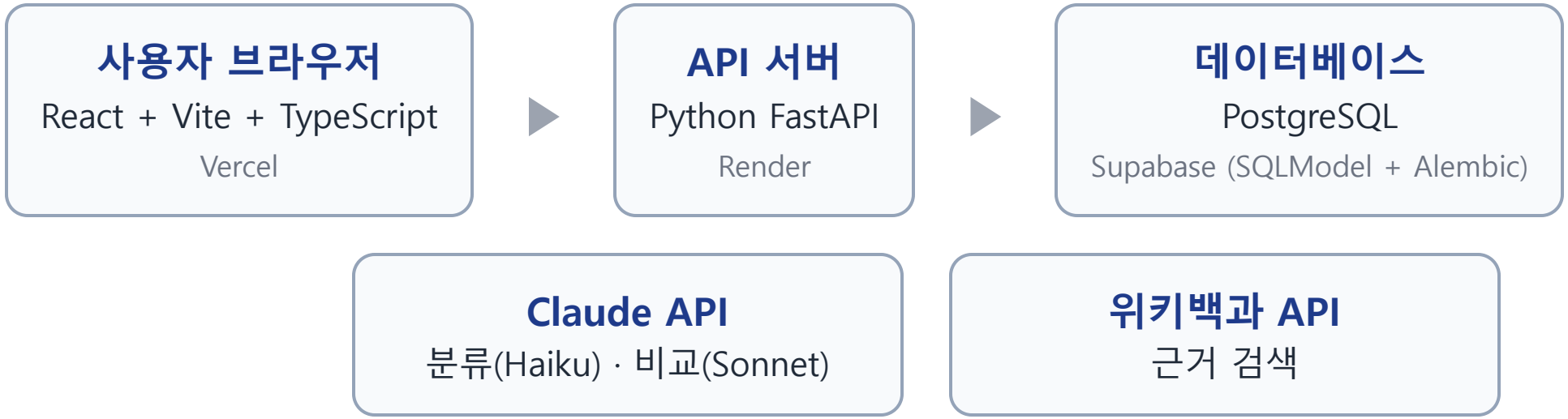
▶

비교
Claude Sonnet이 출처별 입장과 인용문 반환

- 인용문이 **실제 발췌문에 글자 그대로 있는지** 코드가 확인하고, 없으면 그 근거는 낮춘다
- 발췌문이 주장과 다른 대상이나 시점을 말하면(**같은 범위인지** 검사) 근거로 쓰지 않는다
- 검색 실패는 전체 실패가 아니라 해당 문장만 "검색 실패"로 표시

구조

프론트와 백엔드를 분리해 무료로 배포했다



- 비밀값은 환경변수로만 관리하고 GitHub에는 올리지 않는다
- 로컬과 운영 설정 분리, 운영 DB는 개발 DB와 별도 프로젝트

운영 안전

비용, 개인정보, 남용을 막는 장치

비용 제한

IP당 분당 3회, 24시간 20회, 전체 24시간 60회, 동시 분석 3개. 분석 1회 약 $0.03(실측, 5문장)

개인정보

저장 기본값은 **꺼짐**. 저장해도 30일 후 자동 삭제. 로그인 대신 익명 토큰의 **해시만** 저장

외부 전송 고지

입력한 글이 Anthropic(AI)으로, 검색어가 위키백과로 전송된다는 사실을 화면에 안내

- IP는 salt를 섞은 해시로만 저장, 서버 오류 메시지에 내부 정보 노출 안 함
- 잘못된 입력은 호출 횟수를 소모하지 않는다

품질

핵심 원칙을 테스트로 고정했다

182
백엔드 테스트
커버리지 약 98% (11단계 시점 측정)

90
프론트엔드 테스트
계약·문구·접근성

- **문구 가드**: 상태 이름에 "참/거짓/정확/틀림"이 들어가거나 확정 문장이 나오면 테스트 실패
- **계약 테스트**: 백엔드가 내보내는 상태·오류 코드가 프론트에 모두 처리되는지 소스에서 직접 확인
- 테스트가 실수로 실제 Claude/위키백과를 호출하면 실패하도록 차단
- 코드를 일부러 망가뜨려 테스트가 실제로 잡는지 확인(일회성 변이 시험)

모델 선택

평가해서 모델을 나눠 썼다

단계	결과	선택
분류 (확인 대상 구분, 검색어)	Haiku와 Sonnet이 동일(재현율 93/93, 특이도 21/21), Haiku가 약 19배 저렴	Haiku 5.5
근거 비교	Sonnet 58/58, Haiku(medium) 56/58, Haiku(low) 55/58 + 결과가 회차마다 흔들리고 잘못된 반박 1건	Sonnet 5.5

> 한계: 평가 문장은 38개(정확도 대상 29개)로 적고, 정답은 작성자가 직접 정했습니다. 작은 차이는 통계적으로 의미가 없어 **확정이 아닌 현재 기본값**이며 환경변수로 바꿀 수 있습니다.

개발 과정

실제로 부딪힌 문제와 해결

문제	해결
연도만 다른 문서를 "상충"으로 잘못 판단	같은 대상·시점인지 확인하는 항목 추가, 회귀 테스트
위키백과 429(요청 과다)로 검색 실패 다발	속도 조절, 쿨다운, 1시간 캐시, 검증 문장 10개로 제한 (10/10 성공)
서버 내부 오류(500) 응답에 CORS 헤더 없음	CORS 안쪽에서 처리하는 미들웨어 (테스트가 발견)
IP 헤더를 바꿔 호출 제한 우회 가능	프록시 헤더 처리 방식을 측정해 운영 실행 옵션 고정
배포 후 DB 동시 접속 한도 오류(503)	Transaction pooler(6543)로 전환, 부하 시험으로 확인

한계와 다음 단계

지금의 한계를 솔직하게

한계

· 근거가 위키백과 문서 앞부분(약 400자)뿐이라 최신 사건·세부 통계는 "근거 부족"이 많음
· 평가 세트가 작고 정답을 직접 정함
· 운영(Render)에서 IP별 분당 제한이 의도대로 걸리는지 확인 중
· 무료 서버라 첫 요청이 느릴 수 있음, 연달아 분석하면 약 28초
· CI는 GitHub에서 아직 실행해 보지 못함

다음 단계

· 회원가입/로그인, 익명 기록을 계정으로 이전
· 검색 출처 확장(뉴스, 공식 문서 등)과 본문 더 깊이 비교
· 평가 세트 확대와 사람 검수
· 브라우저 자동화 테스트 추가

이 서비스는 "정답을 알려 주는 도구"가 아니라 **"어디를 다시 확인할지 알려 주는 도구"**입니다.

감사합니다

Answer Auditor

검색된 출처 기준의 참고 정보.
최종 판단은 원문 확인으로.

데모 https://ai-verify-deliverability.vercel.app
코드 https://github.com/rhdiddlwpffl/AI_Verify_deliverability

